

Mathematical Derivations and Code Mappings for BrainKAN Explainability

Xin Qi

October 2026

Abstract

This document outlines the formal mathematical framework for the BrainKAN explainability project, providing one-to-one mappings between the theoretical derivations and their concrete implementations in the Python codebase.

1 Edge-Specific KAN Convolution Architecture

1.1 Mathematical Formulation

Let the task-fMRI functional brain network be represented as a directed graph $G = (V, E)$, where V represents the brain regions of interest (ROIs) and E represents the edges (functional connectivities). For a given layer l , the node features are updated via:

$$h_j^{(l+1)} = \sum_{i \in \mathcal{N}(j)} \Phi_{ij}^{(l)}(h_i^{(l)}) \quad (1)$$

Unlike standard Graph Neural Networks (GNNs) that use a shared weight matrix, BrainKAN parameterizes Φ_{ij} uniquely for every existing edge $(i, j) \in E$ using B-splines.

For a scalar input x , the edge-specific function is defined as:

$$\Phi_{ij}(x) = W_{ij}^{base} \text{SiLU}(x) + \text{spline}_{ij}(x) = W_{ij}^{base} \text{SiLU}(x) + \sum_{k=1}^K w_{ij,k} B_k(x) \quad (2)$$

where $B_k(x)$ are the B-spline basis functions defined recursively over a uniform grid, and $w_{ij,k}$ are the learnable spline coefficients specifically allocated for the edge (i, j) .

The updated node features are:

$$h_i' = R(h_i) + \sum_j A_{ij} \Phi_{ij}(h_j) \quad (3)$$

where R is an optional residual connection.

1.2 Code Implementation

File: `models/kan.py`

Class/Function: `EdgeSpecificKANConv.forward()` and `_b_spline_basis()`

The recursive B-spline basis evaluation is centrally implemented in `_b_spline_basis()`, which is imported throughout the repository to guarantee numerical consistency (e.g., in `02_q3b_common_reference_geometry.py`).

2 Q1: Function Characterization and Deviation-from-Linearity (DFL)

2.1 Mathematical Formulation

To characterize whether the learned edge function Φ_{ij} is actually nonlinear, we evaluate it over a continuous domain $T = [t_{min}, t_{max}]$ and compute the Deviation-From-Linearity (DFL). Let $y(t) = \Phi_{ij}(t)$.

We fit a simple linear regression $\hat{y}(t) = at + b$. The DFL is defined as $1 - R^2$, where:

$$R^2 = 1 - \frac{\sum_{t \in T} (y(t) - \hat{y}(t))^2}{\sum_{t \in T} (y(t) - \bar{y})^2} \quad (4)$$

$$\text{DFL} = 1 - R^2 = \frac{\sum_{t \in T} (y(t) - \hat{y}(t))^2}{\sum_{t \in T} (y(t) - \bar{y})^2} \quad (5)$$

A higher DFL indicates that the edge function deviates significantly from a pure linear transformation.

2.2 Code Implementation

File: 01_q1_q2_q3a_naive_analysis.py

Function: compute_function_signatures()

The code computes `ss_res` (residual sum of squares) and `ss_tot` (total sum of squares) to derive R^2 , and then outputs $\text{DFL}_e = 1 - R^2$. The edge-level DFL is averaged across output channels: $NL_e = \frac{1}{16} \sum_o NL_{e,o}$.

3 Q2: Function Taxonomy

3.1 Mathematical Formulation

To determine if edge functions group into a discrete taxonomy of biological computations, we standardize the function signatures and apply Ward hierarchical clustering for $k = 2, \dots, 7$. Cluster quality and fold-level cross-validation stability are measured using Silhouette and Adjusted Rand Index (ARI):

$$\text{Silhouette} \approx 0.18 - 0.19, \quad \text{ARI} \approx 0 \quad (6)$$

This indicates no reproducible small- k discrete taxonomy is detected across folds.

3.2 Code Implementation

Files: 01_q1_q2_q3a_naive_analysis.py (Real) and 05_q2_null_taxonomy.py (Null).

4 Q3A: Naive Curvature Remodeling

4.1 Mathematical Formulation

The naive curvature effect (ΔC) assesses whether the second derivative of the edge function changes across task conditions ($c \in \{0BK, 2BK\}$). For an edge e , the condition-specific naive curvature is calculated over the subject-specific input distribution $X^{(c)}$:

$$C_e(x) = \frac{1}{16} \sum_{o=1}^{16} |f''_{e,o}(x)| \quad (7)$$

The shift in curvature is then:

$$\Delta C_e = C_e(x_j^{2BK}) - C_e(x_j^{0BK}) \quad (8)$$

Note: As discussed, ΔC_e is mathematically confounded by kinematic shifts. By Taylor expansion around a baseline operating point x_0 , the apparent curvature change is driven by the shift in input Δx :

$$\Delta C \approx C'(x_0) \cdot \Delta x + \frac{1}{2} C''(x_0) \cdot \Delta x^2 + \dots \quad (9)$$

This demonstrates why $\Delta C^{\text{real}} \approx \Delta C^{\text{null}}$ under a label-shuffled null, representing an identifiability failure of the naive estimand.

4.2 Code Implementation

File: 01_q1_q2_q3a_naive_analysis.py

The script evaluates the second derivative using PyTorch's `torch.autograd.grad` on the actual task-specific inputs.

5 Q3B: Common-Reference Null Calibration

5.1 Mathematical Formulation

To eliminate the input-regime confounder, we evaluate the learned geometry over a fixed, condition-independent reference grid x_j^{ref} , constructed via quantiles of the pooled training distribution:

$$x_j^{ref} = Q_{\text{pooled training},j}(q) \quad (10)$$

The intrinsic structural nonlinearity (using DFL as the test statistic G_e) is computed for the real model (G^{real}) and compared against $R = 100$ null models (G^{null}):

$$G_e = \frac{1}{16} \sum_o (1 - R_{e,o,\text{ref}}^2) \quad (11)$$

The test statistic measuring excess geometry is:

$$\Psi_e = G_e^{real} - E[G_e^{null}] \quad (12)$$

Statistical significance (yielding 0/240 FDR) is determined by whether G_e^{real} lies in the extreme tail of the empirical null distribution: $\{G_e^{null,1}, \dots, G_e^{null,R}\}$.

5.2 Code Implementation

File: 02_q3b_common_reference_geometry.py

Models are trained identically under true labels vs. subject-level swapped labels. All functions are probed strictly on $X_{ref} = [-8, 12]$.

6 Q3C: Held-Out Calibrated Effective Gain

6.1 Mathematical Formulation

Because Q3B found no excess structural nonlinearity, we test for simpler computational changes: changes in the Effective Gain (the first derivative). To ensure unbiased estimation, we use a fixed condition-independent reference grid X_{ref} , strictly computed from the *training fold* to prevent data leakage.

$$G_{e,c} = \frac{1}{16} \sum_o \mathbb{E}_{x \sim P_c} [|f'_{e,o}(x)|] \quad (13)$$

Since $f'(x)$ is evaluated via numerical differentiation:

$$f'(x) \approx \frac{f(x + \epsilon) - f(x - \epsilon)}{2\epsilon} \quad (14)$$

The paired difference in effective gain for a subject is:

$$dG_i = G_i^{2BK} - G_i^{0BK} \quad (15)$$

We test the hypothesis that the paired differences from the real model differ from the null:

$$H_0 \Rightarrow (dG_i^{real}, dG_i^{null}) \stackrel{d}{=} (dG_i^{null}, dG_i^{real}) \quad (16)$$

Let $T_e^{real} = dG_i^{real} - dG_i^{null}$. The permutation p-value with R sign-flips is:

$$p_e = \frac{1 + \sum_r I(|T_e^{null,r}| \geq |T_e^{real}|)}{R + 1} \quad (17)$$

$$d_z = \frac{\mu_{\Delta G}}{\sigma_{\Delta G}} \quad (18)$$

where $\mu_{\Delta G}$ and $\sigma_{\Delta G}$ are the mean and standard deviation of $dG_{e,s}^{real} - dG_{e,s}^{null}$ across all test subjects.

6.2 Code Implementation

File: 04_q3c_effective_gain.py

Functions: `evaluate_Q3C()` and the subsequent statistical testing block.

Cohen's d_z is explicitly calculated as `observed_mean / np.std(diff)` and saved in `q3c_final_table.csv`.

7 Synthetic Identifiability Benchmark

7.1 Mathematical Formulation

To define the identifiability boundary, we compare the measured effective gain ΔG across synthetic scenarios: baseline (S_0), input shift (S_1), and gain modulation (S_2).

$$S_1(\text{input shift}) : \Delta G = -0.087 \pm 0.13 \quad (19)$$

$$S_2(\text{gain modulation}) : \Delta G = -0.082 \pm 0.13 \quad (20)$$

$$S_1 \approx S_2 \quad (21)$$

This mathematically proves that input shift and gain modulation are confusable under the tested Q3C estimator. To ensure the estimator can detect true structural remodeling when present, S_5 tests a strong true structural change via a common-support comparison distance D :

$$D_{S5} = 3.574 \pm 0.75 \gg D_{S0} = 0.153 \pm 0.25 \quad (22)$$

Rejecting $H_0 : S_5 = S_0$ proves the estimator responds to the tested remodeling regime.

7.2 Code Implementation

File: 03_synthetic_identifiability_benchmark.py

Simulates S0-S5 using synthetic 1D splines.